Healthcare of Bangladesh: An Incremental Sectoral Data Integration Model

Database Systems Project

Scrum Level-1

Scrum Group-3

Report submitted August 2026

A project submitted to **Professor Dr. Rudra Pratap Deb Nath**, Department of Computer Science and Engineering, Chittagong University (CU), in partial fulfillment of the requirements for the Database Systems Lab course. The project is not submitted to any other organization simultaneously.

Group Details

Sl.	Name	Student ID	Session	Signature
1	Md. Alamin Molla	24701076	2023–2024	
2	Sheikh Showkotul Islam Shakib	24701041	2023–2024	
3	Ratul Hasan Nipu	23701060	2022–2023	
4	Jahid Alam Ridoy	23701039	2022–2023	

Table 1: Details of the Project Group

Supervisor: Professor Dr. Rudra Pratap Deb Nath

Supervisor Approval: _____

Mentor: Miskatul Anwar

Mentor's Signature:_____

Date: _____

Preface

This report documents the work carried out at the Department of Computer Science and Engineering, University of Chittagong, during August 2026. The title of the project is *Health Data Integration, Database Design and Entity Relationship Diagram Development using Government Health Reports of Bangladesh*, undertaken as part of the Database Systems Lab course.

This project is part of a larger collaborative effort in which all student groups are working on the same overarching system: **MODELBD**, an integrated database system covering seven distinct data sectors, including Agriculture, Healthcare, Energy, and others. The Scrum of Scrum (3-level) approach is adopted to develop MODELBD. At the bottom-most layer, each group is assigned one sector. Our group is assigned the **Healthcare** sector.

In short, the problem we address is that Bangladesh's health-related information is scattered across many disconnected government documents; our response is to connect that scattered information into a single, structured database. Our work involves collecting government health reports of Bangladesh, processing and extracting structured data from Portable Document Format documents, designing a normalized relational database, and developing conceptual as well as physical Entity Relationship Diagrams.

Md. Alamin Molla

Sheikh Showkotul Islam Shakib

Ratul Hasan Nipu

Jahid Alam Ridoy

Contents

List of Tables

Abstract

Motivation

Bangladesh generates a vast amount of health-related data through government ministries, hospital registries, training programs, and annual health bulletins. However, this data remains scattered across hundreds of unstructured PDF documents, making it inaccessible for real-time analysis, planning, and policy-making. This project is motivated by the urgent need to transform these fragmented health datasets into a structured, searchable, and central system. Specifically, the development of an automated disease surveillance and resource forecasting dashboard drives this effort. By digitizing regional disease incidence rates alongside hospital capacity metrics, such as bed availability, ICU usage, and medical supplies, the system provides interactive visualizations and real-time query capabilities. This allows policymakers and healthcare administrators to monitor localized outbreaks instantly, optimize resource distribution, and make data-driven clinical and operational decisions.

Problem Definition: The core problem addressed in this project is the unavailability of machine-readable, structured health data from Bangladesh government sources. Key entities such as hospitals, diseases, health programs, budgets, training activities, cancer registries, and health workers are documented in reports but are not available in a queryable database format.

Limitations of Current Solutions: Currently, health stakeholders must manually browse through numerous Portable Document Format reports to find relevant information. There is no centralized database that integrates data across entities such as `Patient`, `Hospital`, `Disease`, `HealthProgram`, `Budget`, `Training`, and `CancerRegistry`. Inconsistencies in naming conventions and reporting formats across documents further hinder data reuse. This directly connects back to the problem stated above: fragmented documents cannot, on their own, answer cross-cutting questions that stakeholders actually need answered.

Our Approach: We adopt an incremental, sprint-based development process using Scrum methodology. The approach involves systematic data discovery, automated Portable Document Format collection, table extraction, data cleaning and transformation, normalization, MySQL database implementation, and Entity Relationship Diagram development.

Our Solution: The developed system provides a fully normalized MySQL database comprising 21 relational tables, logically connected to represent the major entities of

the Bangladesh health sector. A conceptual (logical) Entity Relationship Diagram with 21 entities and a physical database Entity Relationship Diagram are also produced, along with automated Extract-Transform-Load pipeline scripts intended for deployment to a version-controlled repository.

Evaluation: The solution is evaluated by running SQL (Structured Query Language) aggregate queries, validating record counts, checking foreign key integrity, and cross-referencing extracted data with original Portable Document Format source documents. This evaluation confirms a full, end-to-end connection between the source reports and the final database.

Significance, Limitations, and Future Work: This project demonstrates that unstructured government health data can be transformed into a structured, queryable format using systematic engineering approaches, giving health administrators and researchers a single point of access instead of hundreds of disconnected PDFs. A current limitation is that the database is built only from the reports collected during Sprint 1 and relies on manually verified source documents, so it does not yet cover the full range of Bangladesh's health reporting. Future work includes extending coverage to additional health sectors, automating data updates from government portals, and building analytical dashboards on top of the database.

Chapter 1

Introduction

We are collecting information from various health-related government sources and organizing it into a single, structured, integrated database. The purpose of this project is to take scattered health information and organize it so that related information can be easily stored, retrieved, and queried. This report describes the project's background, problem, proposed system, development process, and related aspects, as our contribution to MODELBD — a large-scale, integrated Bangladesh sectoral database system developed collaboratively using the Scrum of Scrum methodology as part of the Database Systems Lab course at the Department of Computer Science and Engineering, Chittagong University.

The objective of this course is to develop an integrated health information database by applying the theories, methodologies, tools, and technologies learned in the Database Systems Lab course. Our group is responsible for the **Healthcare** sector.

1.1 Background and Motivation

Current State: Bangladesh's health sector produces a substantial volume of documentation annually through its Ministry of Health and Family Welfare, hospitals, and various health programs, covering disease surveillance, health information systems, and public health programs. These documents include annual health bulletins, hospital cancer registry reports, training reports, operational plans, budget documents, and statistical yearbooks. This kind of health information system is meant to support data analysis, interpretation, dissemination, and evidence-based planning and decision-making, but that intended use depends on the underlying information being organized and connected.

Problems Faced by Stakeholders: Despite the richness of this information, it remains locked inside Portable Document Format documents that are not machine-readable, and similar information is spread across many separate reports in inconsistent formats. Health administrators, researchers, and policymakers who need data-driven insights are forced to manually search through and cross-reference hundreds of documents to find and connect related information, which is both time-consuming and error-prone.

Our Approach: Rather than simply collecting these documents, our approach is to transform their contents into a structured database. We extract information from the source health documents, identify the relevant entities and their attributes, identify the relationships between those entities, and use this to build a conceptual Entity Relationship model, a relational model, and finally a normalized, implemented database.

Significance: Structuring this information matters because it turns fragmented, manually-searched documents into something that can be queried directly: related information becomes connected, retrieval becomes efficient, and the result provides a structured foundation for future analysis. Converting scattered health-related information into a structured and queryable database makes information retrieval, integration, and future analysis easier for the stakeholders described above.

1.2 Problem Statement

The problem addressed by this project is to identify, organize, and integrate the relevant health-related entity types, their attributes, and relationships from multiple heterogeneous government health documents into a structured, normalized relational database. Specifically, entities including `Patient`, `HealthFacility`, `Disease`, `HealthWorker`, `CancerCase`, `Vaccination`, `Laboratory`, `MaternalHealth`, `AdministrativeRegion`, and `PopulationGroup` are documented in government reports but lack structured, interconnected database representations. This project aims to design and implement a normalized database that represents these entities and the relationships among them in a consistent relational structure.

1.3 System Definition

> *A computerized system used to integrate and manage health-related information collected from multiple government documents, by organizing entities, attributes, and relationships — with well-defined entity relation types between them — into a normalized MySQL database accessible through standard SQL queries and supported by automated Extract-Transform-Load pipeline scripts, for efficient storage, retrieval, and querying.*

1.4 System Development Process

The system is developed using an agile Scrum methodology organized in sprints. The development lifecycle includes the following phases:

1. **Data Discovery:** Identification and inventory of relevant government health documents.

2. **Data Acquisition:** Automated collection of Portable Document Format reports using Python scripts.
3. **Data Extraction:** Table and text extraction from Portable Document Format documents into Comma-Separated Values format.
4. **Data Cleaning and Transformation:** Standardization, deduplication, and normalization of extracted data.
5. **Entity Identification:** Identification of the major entities, and their attributes, present across the health domain.
6. **Relationship Identification:** Identification of the entity relation types (connections) between the identified entities.
7. **Conceptual Design:** Construction of the conceptual Entity Relationship model from the identified entities and relationships.
8. **Logical Design:** Conversion of the Entity Relationship model into a relational schema.
9. **Normalization:** Normalization to Third Normal Form to reduce redundancy and dependency anomalies.
10. **Database Implementation:** MySQL table creation and data loading.
11. **Validation:** SQL-based verification of data accuracy and integrity.
12. **Entity Relationship Diagram Development:** Conceptual (logical) and physical Entity Relationship Diagram generation.

1.5 Organization

Section 2 describes how the project resources and team roles are organized. Section 3 presents the Sprint 1 activities in detail, covering data discovery, acquisition, extraction, cleaning, database design, and Entity Relationship Diagram development. Section 4 explains the deployment and maintenance of the project repository. Section 5 reflects on intra- and inter-group collaboration. Section 6 offers comments on course pedagogy. Finally, Section 7 presents the conclusion and future work.

Chapter 2

Project Structure and Management

This chapter describes how our team is organized and how we plan, divide, track, and manage the project's work.

2.1 Resource Management

The project's resources are organized into three broad categories. The **human resources** are the four group members and the course supervisor. The **information resources** are the collected government health documents, extracted datasets, and reference materials that form the raw input to the database. The **software resources** are the tools used for version control, database development, and documentation, listed in Table 2.2. The project team consists of four members, and responsibilities are divided among them to maximize parallel progress while ensuring regular coordination, with each member owning a distinct area of the pipeline while remaining accountable to the shared Product Backlog described in Section 2.5.

2.2 Team Roles and Responsibilities

Assigning a distinct role to each member also gives the project accountability: if a particular task needs to be traced back to a person, for example, who designed the Entity Relationship diagram, or who maintained the Portable Document Format repository, the responsibility is clear from the table below.

Member	Role	Main Responsibilities
Md. Alamin Molla	Team Leader	Coordinates sprint planning, manages task distribution, and oversees database normalization and overall project progress
Sheikh Showkotul Islam Shakib	Data Collector	Identifies and collects government health reports, verifies document authenticity, and maintains the Portable Document Format repository
Ratul Hasan Nipu	Entity Relationship Diagram Designer	Designs the conceptual (logical) Entity Relationship Diagram (21 entities) and the physical database Entity Relationship Diagram based on the normalized schema
Jahid Alam Ridoy	Script Writer	Develops Python scripts for automated Portable Document Format downloading, table extraction, data cleaning, and MySQL import utilities

Table 2.1: Team Roles and Responsibilities

2.3 Tools Used

Each tool below serves a specific purpose in the workflow: GitHub provides version control and a shared history of the codebase so that changes can be tracked and previous versions recovered; Python, Pandas, and Regular Expressions handle the data-processing side (downloading, extracting, and cleaning); MySQL is the target database system; and Mermaid/LaTeX TikZ are used to visualize the Entity Relationship diagrams once the schema is finalized.

Tool/Technology	Purpose
GitHub	Version control, code repository, Extract-Transform-Load pipeline deployment
Python	Portable Document Format downloading, data extraction, data cleaning scripts
Pandas	Data manipulation and transformation
MySQL	Relational database implementation
Mermaid / LaTeX TikZ	Entity Relationship Diagram visualization
Linux Terminal	File management, bulk processing, command-line tools
Regular Expressions	Pattern matching during data cleaning

Table 2.2: Tools and Technologies

2.4 Scrum Structure

Scrum is an agile project-management framework in which a large project is broken into smaller tasks, and progress is tracked regularly through short cycles of work called sprints. Briefly, our project follows a three-level Scrum of Scrum hierarchy. At Level 1, our group independently develops the Healthcare sector database. Level 2, in which our group's outputs will be integrated with the other sector groups, and Level 3, in which the combined sectors form the complete MODELBD system, have not yet taken place at the time of writing this report.

Within our group, we maintain a **Product Backlog** listing all major tasks (Section 2.5). Each sprint begins with a **Sprint Planning** meeting, in which a subset of the Product Backlog is selected as that sprint's Sprint Backlog; progresses through **Daily Stand-ups**, where progress and blockers are discussed; and concludes with a **Sprint Review**, where completed work is checked against the sprint's goals.

2.5 Product Backlog Summary

The Product Backlog is the prioritized list of all tasks, features, and requirements needed to complete the project; without it, task ownership, sequencing, and completion status would be difficult to track across four people. The Product Backlog for Sprint 1 includes the following items:

1. Identify and inventory health data sources from government portals
2. Develop automated Portable Document Format downloader

3. Extract tables from Portable Document Format reports into Comma-Separated Values format
4. Clean and standardize extracted datasets
5. Design normalized database schema
6. Implement MySQL database with 21 tables
7. Validate data accuracy using SQL queries
8. Develop conceptual (logical) Entity Relationship Diagram (21 entities)
9. Generate physical database Entity Relationship Diagram
10. Document all processes

Chapter 3

Sprint 1

Following our Scrum structure, this chapter documents Sprint 1 as a complete record of what was planned, what we did, the problems we faced, how we solved them, and what we produced. The **Sprint Goal** for Sprint 1 is to take the Healthcare sector from raw, unstructured government reports to a validated, normalized MySQL database with accompanying Entity Relationship diagrams. Sprint 1 covers the complete development cycle from data discovery to database implementation and Entity Relationship Diagram generation over six weeks.

3.1 Sprint Backlog and Overview

Week	Task	Duration	Outcome
Week 1	Data Collection & Source Identification	5–6 Days	100+ reports inventoried
Week 2	Automated Portable Document Format Collection	4–5 Days	Automated repository created
Week 3	Portable Document Format Processing & Table Extraction	6–7 Days	Hundreds of Comma-Separated Values files generated
Week 4	Data Cleaning & Transformation	4–5 Days	Database-ready CSVs produced
Week 5	Database Design & Normalization	3 Days	Fully normalized schema
Week 6	MySQL Implementation, Validation & Entity Relationship Diagram Development	2–3 Days + 5 Days	21 tables created, Entity Relationship Diagram and validated database produced

Table 3.1: Sprint 1 Task Summary

3.2 Week 1: Data Collection and Source Identification

The sprint commences with the identification of reliable health-related datasets from Bangladesh Government publications. The initial expectation is that structured datasets

would be readily available in Excel or database format. However, the data collection stage reveals numerous challenges.

3.2.1 Challenges Faced

- Health information is distributed across multiple government websites, annual health bulletins, hospital cancer registry reports, training reports, and statistical yearbooks.
- Many document download links are inactive or broken.
- Most datasets exist only as Portable Document Format documents rather than machine-readable files.
- Similar information appears across multiple documents with different naming conventions, reporting periods, and classifications.
- The same health program is sometimes referred to by its full name and sometimes by an abbreviation, causing confusion during integration.

3.2.2 Approach

To address these issues, we manually review available reports, verify document authenticity, categorize documents by topic, and prepare an organized inventory before proceeding.

3.2.3 Outcome

- More than 100 health-related reports are identified.
- Relevant datasets are shortlisted.
- A source inventory is prepared for automated collection.

3.3 Week 2: Automated Portable Document Format Collection and Repository Creation

Once relevant reports are identified, manually downloading each file becomes impractical. Therefore, automated Portable Document Format collection is implemented using Python.

3.3.1 Challenges Faced

- Different websites use different web-address structures and file naming conventions.

3.2.4 Data Discovery and Search Strategy

Data discovery was performed before downloading or transforming any file. Its purpose was to identify authoritative Bangladesh health material that could support the fixed 21-entity database scope, while preserving enough metadata to explain why a source was selected. The team treated a catalog record, a downloaded document, an extracted table and a database row as four different evidence units. A source could therefore be useful for background or schema design even when it could not be loaded as a physical row.

3.2.4.1 Scope and source selection

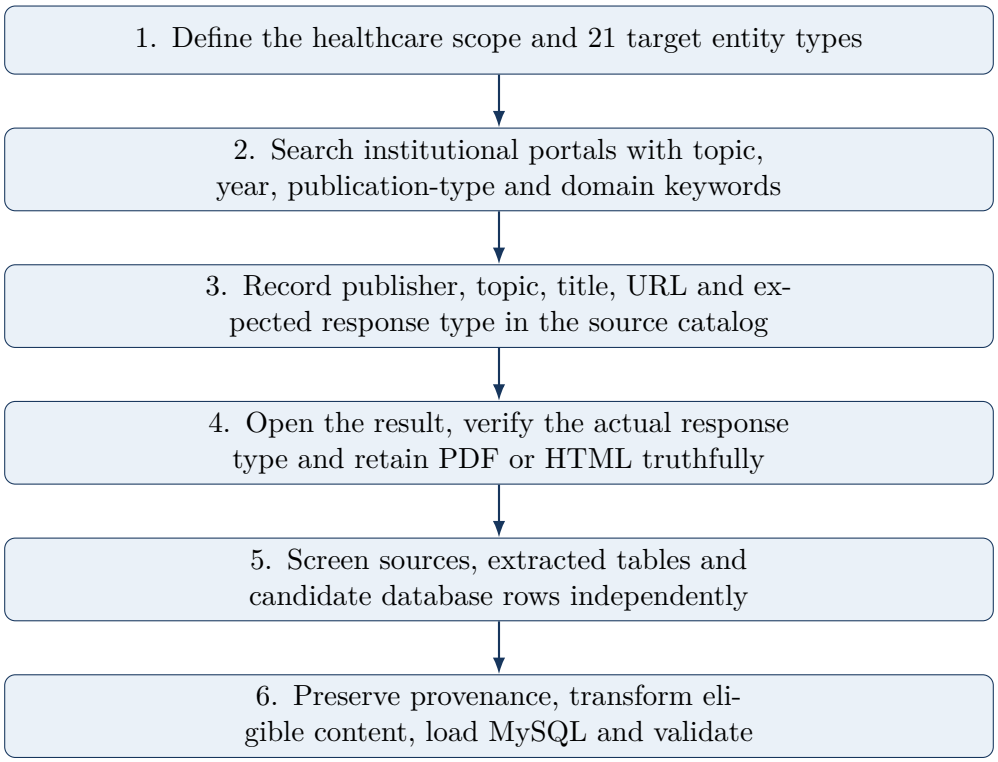
The search focused on health facilities, workforce, hospital resources, laboratories, population, communicable and non-communicable diseases, maternal and newborn health, nutrition, vaccination and digital-health services. Priority was given to the Directorate General of Health Services (DGHS), Ministry of Health and Family Welfare (MoHFW), UNICEF, WHO, USAID DHS Program, NICVD and NICRH. For each candidate, the team recorded publisher, subcategory, dataset or report title, URL and expected file type in `health_data.xlsx`. The original grouped worksheet is preserved as `Health`; `Health_Catalog_Clean` provides 112 machine-readable records with no blank field.

3.2.4.2 Search method and keywords

Searches combined a Bangladesh location term, a health topic, a publication type and, where useful, a publisher or domain filter. The exact browser-history query log was not retained; the representative terms below are reconstructed from the catalog categories, dataset titles and source URLs and describe the searches used to locate the collected material.

Keyword group	Representative search terms and query patterns
General	Bangladesh health data; Bangladesh health statistics; Bangladesh health indicators; Bangladesh health bulletin PDF
Disease surveillance	COVID-19 Bangladesh; dengue status report; measles outbreak Bangladesh; measles-rubella bulletin; HIV/AIDS report; tuberculosis annual report; HMPV; infectious-disease surveillance
Maternal, child and nutrition	maternal mortality survey; pregnancy history; newborn health; immunization; vaccination; child disability; adolescent nutrition; micronutrient survey; anthropometry
Facilities and workforce	hospital and health-facility data; hospital beds; laboratory; HRH data sheet; doctors; nurses; health workforce; designation; e-Health; telemedicine
Specialized institutes	NICRH cancer registry; Cancer Journal Bangladesh; NICVD cardiovascular disease journal; Bangladesh oncology yearbook
Domain filters	`site:dghs.gov.bd filetype:pdf`; `site:who.int Bangladesh`; `site:unicef.org/bangladesh`; `site:dhsprogram.com`; `site:nicrh.gov.bd`; `site:nicvd.gov.bd`

3.2.5 Discovery Workflow and Inventory



3.2.5.1 Discovery inventory

Source	Records	Source	Records
DGHS	61	UNICEF	15
WHO	11	NICRH	10
Ministry of Health & Family Welfare	9	NICVD	4
USAID DHS Program	2	**Total**	**112**

The catalog identifies 108 intended PDF records and four HTML web pages. The current repository preserves 61 files with valid PDF signatures under `pdfs/` and four verified web pages under `source_pages/`. File type is determined from the downloaded response rather than the URL extension. Consequently, "cataloged", "successfully archived", "extractable" and "loaded" must not be reported as the same count.

3.2.5.2 Metadata captured during discovery

For every catalog row, the clean worksheet carries the original row number, category, publisher, subcategory, dataset name, source URL, source type and a quality note. The source URL provides the publication location; the dataset title and retained document provide survey or coverage years where available. Team collection dates are reported separately because a publication year is not a download date.

3.2.6 Inclusion and Exclusion Criteria

Screening level	Inclusion criteria	Exclusion criteria
Source	Identifiable health-sector publisher; Bangladesh relevance; meaningful title and URL; topic relevant to a target entity or reference domain; response retained in its truthful file type.	Broken, empty or non-health response; unverifiable title or origin; material outside the healthcare scope; duplicate alias counted as another independent source; HTML presented as a PDF.
Extracted table	At least 5 rows and 2 columns; readable tabular structure; unique content hash; metadata retaining original document, page, dimensions and extracted filename.	Fewer than 5 rows or 2 columns; all-empty rows or columns; exact duplicate hash; extraction failure; narrative fragments without a stable table structure.
Database row	Maps to the fixed 21-entity schema; every mandatory attribute can be populated; foreign-key parents exist; value is source-backed or explicitly labelled as synthetic demonstration data.	Missing mandatory attribute; orphan or duplicate; semantic mismatch; unsupported granularity; source aggregate represented falsely as an individual record.

The sanctioned-bed total in Health Bulletin 2023 illustrates the granularity rule. The source gives aggregate capacity, while the HospitalBed table requires a separate BedID, BedType and Status for every record. The aggregate was therefore not converted into fabricated individual beds.

3.2.7 Screening Outcome and Evidence

Evidence stage	Count	Repository evidence
Cataloged candidate records	112	`health_data.xlsx`
Verified archived PDFs	61	`pdfs/`, `SOURCE_ARCHIVE_STATUS.md`
Verified archived HTML pages	4	`source_pages/`
Extraction/metadata source folders	68	`extracted_tables/`, `metadata/`
Extracted CSV tables	3,797	`reports/table_catalog.csv`
Cleaning jobs completed	3,797	`reports/cleaning_summary.csv`
Exact duplicate tables flagged	572	`reports/duplicate_tables.csv`
Canonical database rows	495	`sql/health_db_21_tables_with_data.sql`

These numbers describe different units, not one clinical sample size. The final 495-row database is a reproducible demonstration snapshot. Excluded source material remains in the discovery or archive evidence when appropriate, while inaccessible, duplicate, overly coarse or schema-incompatible content is not presented as source-backed database data.

3.2.8 Source Coverage and Collection Metadata

The table separates three dates that must not be confused: the period represented in a dataset, the publication or release period, and the team's acquisition window. A range shows the earliest and latest year evidenced across that source's cataloged material; it does not imply that every intervening year is available. Exact HTTP download timestamps were not logged, so month-level acquisition dates are identified as team-reported records.

Source	Data coverage / survey year	Publication / release evidence	Collection / access	Data represented
UNICEF	2011–2026	Survey 2011–12; reports and programmes 2014–2026	April 2026 (team record)	Measles, HIV, childhood disability, vaccination and nutrition.
WHO	1970–2026	Historical table from 1970; Health System Review 2015; mental-health report 2025; measles notice 23 April 2026	Late April 2026 (team record)	Tuberculosis, dengue, measles, health indicators, mental and maternal health.
DGHS	2007–2026	Health Bulletins 2007–2024; outbreak and status updates in April–May 2026	April–May 2026	COVID-19, dengue, cancer, HMPV, e-Health, climate, measles, rubella and maternal mortality.
NICVD	1980–2018	Historical coverage recorded in the team's source notes; four cataloged journal links	April 2026 (team record)	Cardiovascular-disease information.
MoHFW	2009–2024	Performance report 2009–10; HRH/workforce publications 2011–2024	April 2026 (team record)	Doctors, health facilities, hospitals, workforce and designations.
USAID DHS Program	2024	Anthropometry report August 2024; pregnancy-history report October 2024	April 2026 (team record)	Pregnancy-history and anthropometry methods.
NICRH	2020–2024	Cancer Journal Bangladesh volumes and Yearbook published 2020–2024	April 2026 (team record)	Cancer, oncology journals and institutional yearbook material.

The year corrections are based on `Health_Catalog_Clean`, retained PDF titles and text, and the two USAID publication title pages. In particular, UNICEF begins with the 2011–12 micronutrient survey, DGHS extends to 2026 updates, USAID is 2024, and NICRH spans 2020–2024.

- Some files contain special characters in their names.
- Incomplete downloads occur due to temporary network interruptions.
- Without proper organization, hundreds of downloaded files would become unmanageable.

3.3.2 Tools Used

Python, Required Library, Operating System and File Handling Modules, Linux Terminal.

3.3.3 Performance

- Small batch download: 10–20 seconds
- Large batch download: 2–5 minutes

3.3.4 Outcome

- An automated Portable Document Format repository is created.
- Hundreds of reports are downloaded successfully.
- Manual effort is reduced significantly.

3.4 Week 3: Portable Document Format Processing and Table Extraction

This is the most challenging phase of Sprint 1. Although the reports appear visually organized, extracting usable data proves extremely difficult due to complex Portable Document Format layouts.

3.4.1 Challenges Faced

- Tables often extend across multiple pages, creating the impression that data is missing.
- Some table entries are broken into several rows during extraction, causing columns to shift.
- Many extracted tables lose their headers.
- Certain pages contain scanned content where traditional extraction methods perform poorly.

3.4.2 Tools Used

Python, Pandas, Comma-Separated Values Processing, Regular Expressions, and Linux command-line tools including `grep`, `sed`, `awk`, `head`, `tail`, `find`, and `wc`.

3.4.3 Performance

- Single Portable Document Format extraction: 2–10 seconds
- Bulk extraction: 5–20 minutes

3.4.4 Outcome

- Hundreds of structured Comma-Separated Values files are generated.
- Cancer registry datasets are reconstructed.
- Missing sections are successfully recovered.

3.5 Week 4: Data Cleaning, Transformation and Categorization

After extraction, the generated Comma-Separated Values files require extensive cleaning before they can be loaded into a database.

3.5.1 Data Quality Issues Identified

- Missing values across multiple fields.
- Duplicate records from overlapping reports.
- Inconsistent spellings and abbreviations of health program names.
- Numeric fields occasionally containing symbols instead of numeric values.
- Multi-line records requiring reconstruction.

3.5.2 Tools Used

Python, Pandas, Regular Expressions, and spreadsheet-based spot checks for manual verification of edge cases.

3.5.3 Outcome

- Clean datasets are produced.
- Standardized naming conventions are established.

- Database-ready Comma-Separated Values files are prepared.

3.6 Week 5: Database Design and Normalization

3.6.1 Normalization

Once clean datasets are available, normalization principles are applied to the physical Patient table to eliminate redundancy and anomalies.

First Normal Form: Each Patient column contains one atomic value, and PatientID uniquely identifies every row.

Second Normal Form: PatientID is a single-column primary key, so partial dependency cannot occur; every non-key attribute depends on the whole key.

Third Normal Form: No non-key Patient attribute determines another non-key Patient attribute. Bed details are not stored in Patient; only BedID is retained as a foreign-key attribute.

3.6.2 Worked Example: Normalizing the Patient Table

This worked example uses only the implemented Patient relation. No columns from any other entity are merged into the normalization tables. The exact physical column order is PatientID, FullName, DateOfBirth, Gender, NationalID, and BedID.

PatientID is the primary key. NationalID is unique and acts as an alternate candidate key. BedID is a populated foreign-key attribute. The first five rows shown on the next page are copied from the final MySQL database dump.

Because the physical Patient table already contains atomic values and a single-column primary key, normalization does not require splitting this entity. Each stage below verifies the same Patient relation against progressively stronger rules.

Functional Dependencies

FD	Functional Dependency
FD1	PatientID -> FullName, DateOfBirth, Gender, NationalID, BedID
FD2	NationalID -> PatientID, FullName, DateOfBirth, Gender, BedID

Table 3.2: Functional Dependencies Used in the Patient Normalization Example

First Normal Form

PatientID	FullName	DateOfBirth	Gender	NationalID	BedID
1	Amena Khatun	1979-01-01	Female	DEMO-NID-000001	1
2	Rahim Uddin	1980-02-02	Male	DEMO-NID-000002	2
3	Nasrin Akter	1981-03-03	Female	DEMO-NID-000003	3
4	Md. Hasan Ali	1982-04-04	Male	DEMO-NID-000004	4
5	Shamima Sultana	1983-05-05	Female	DEMO-NID-000005	5

Table 3.3: Patient - Actual Data in First Normal Form

1NF criterion	Result	Evidence in Patient
Atomic values	Pass	One value in every cell
Unique rows	Pass	PatientID is the primary key
Repeating groups	None	No repeating or multi-valued columns

Table 3.4: First Normal Form Verification for Patient

Description: The displayed records use the exact six Patient columns and actual values from the populated MySQL table. Every field contains one scalar value: dates use one date, Gender uses one value, NationalID uses one identifier, and BedID uses one relationship value. PatientID uniquely identifies each row, while no repeating or multi-valued group is stored inside a Patient cell. The relation therefore satisfies First Normal Form without changing the physical rows.

Second Normal Form

Patient column	Key role	Dependency
PatientID	PK	Determinant of the row
FullName	Non-key	Fully depends on PatientID
DateOfBirth	Non-key	Fully depends on PatientID
Gender	Non-key	Fully depends on PatientID
NationalID	UNIQUE	Alternate candidate key
BedID	FK	Fully depends on PatientID

Table 3.5: Patient Schema in Second Normal Form

2NF test	Patient	Conclusion
Composite primary key?	No	PatientID is one column
Partial dependency?	No	Cannot occur with this key
Second Normal Form?	Yes	All attributes depend on full key

Table 3.6: Second Normal Form Verification for Patient

Description: Second Normal Form requires a relation to be in First Normal Form and requires every non-key attribute to depend on the whole primary key. The implemented Patient table has the single-column primary key PatientID; therefore the key has no proper subset and a partial dependency cannot exist.

FullName, DateOfBirth, Gender, NationalID, and BedID are determined by PatientID. NationalID is also unique and acts as an alternate candidate key, but it does not create a composite-key dependency. Consequently the Patient relation remains unchanged when progressing from First Normal Form to Second Normal Form.

Third Normal Form

Patient column	Key role	3NF dependency
PatientID	PK	Primary determinant
FullName	Non-key	Directly depends on PatientID
DateOfBirth	Non-key	Directly depends on PatientID
Gender	Non-key	Directly depends on PatientID
NationalID	UNIQUE	Alternate candidate key
BedID	FK	Directly depends on PatientID

Table 3.7: Patient Schema in Third Normal Form

Description: Third Normal Form removes transitive dependencies between non-key attributes. Patient stores no BedType, Status, FacilityID, FacilityName, RegionName, or other descriptive facts outside the patient entity. It retains only BedID as a foreign-key value. Thus no non-key Patient attribute determines another non-key Patient attribute, and no additional decomposition is required.

3NF test	Patient	Conclusion
Non-key -> non-key dependency	None	No transitive dependency
All non-key attributes direct?	Yes	Depend on PatientID
Third Normal Form?	Yes	No further split required

Table 3.8: Third Normal Form Verification for Patient

Constraint	Patient column	Role
Primary key	PatientID	Unique row identifier
Alternate key	NationalID	Unique candidate key
Foreign key	BedID	Relationship attribute

Table 3.9: Patient Key Constraints in Third Normal Form

Boyce-Codd Normal Form Verification

Patient attribute	Determinant	Dependency status
PatientID	PatientID	Primary key
FullName	PatientID	Direct dependency
DateOfBirth	PatientID	Direct dependency
Gender	PatientID	Direct dependency
NationalID	PatientID / NationalID	Alternate candidate key
BedID	PatientID	Direct dependency

Table 3.10: Patient Attribute Dependency Summary

Relation	Determinant -> dependent attributes	Superkey?	Verdict
Patient	PatientID -> FullName, DateOfBirth, Gender, NationalID, BedID	Yes	Holds
Patient	NationalID -> PatientID, FullName, DateOfBirth, Gender, BedID	Yes	Holds

Table 3.11: Boyce-Codd Normal Form Verification for Patient

Final table	Primary key	Alternate key	Foreign key	Highest NF
Patient	PatientID	NationalID	BedID	BCNF

Table 3.12: Final Boyce-Codd Normal Form Schema for Patient

Description: Boyce-Codd Normal Form requires every determinant of a non-trivial functional dependency to be a superkey. PatientID is the primary determinant of FullName, DateOfBirth, Gender, NationalID, and BedID. NationalID is an alternate candidate key and also determines the remaining Patient values. Because both determinants are candidate keys, the physical Patient relation satisfies BCNF. BedID being a foreign key does not create a BCNF violation because no Patient attribute is functionally determined by BedID inside this table.

3.6.3 Outcome

Using its exact six implemented columns and database-backed sample values, Patient is verified as 1NF, 2NF, 3NF, and BCNF without introducing any additional normalization table into this worked example.

3.7 Week 6: MySQL Implementation, Validation and Entity Relationship Diagram Development

Twenty-one normalized tables are created and linked using primary keys and foreign keys in MySQL. Key implementation challenges include datatype mismatches, foreign-key dependency ordering, and null-value handling. These are resolved using numbered SQL migrations, Python import utilities, and automated validation queries.

Table No.	Table Name	Description
1	ADMINISTRATIVE _REGION	Administrative divisions/regions of Bangladesh (region name, type, population)
2	POPULATION_GROUP	Population figures for a region, broken down by group
3	HEALTH_FACILITY	Health facility records (type, name) linked to an administrative region
4	HOSPITAL_BED	Bed inventory and status per health facility
5	LABORATORY	Laboratory records linked to a health facility
6	TELEMEDICINE _CENTER	Telemedicine centers offering remote consultations to patients
7	DESIGNATION	Health workforce job designations
8	HEALTH_WORKER	Health worker records, linked to a facility and a designation
9	PATIENT	Core patient records (name, date of birth, gender, National Identification Number)
10	DISEASE	Catalog of tracked diseases with International Classification of Diseases codes
11	DENGUE	Dengue-specific case details (subtype of Disease)
12	COVID19	COVID-19-specific case details (subtype of Disease)
13	MEASLES	Measles-specific case details (subtype of Disease)
14	HIV	HIV-specific case details (subtype of Disease)
15	DIARRHEA	Diarrhea-specific case details (subtype of Disease)
16	CANCER_CASE	Cancer case records (type, stage, test result) tested at a laboratory
17	BIOPSY	Biopsy procedures linked to a cancer case
18	VACCINATION	Vaccine catalog and patient vaccination records
19	MATERNAL_HEALTH	Maternal health records linked to a patient
20	NEWBORN	Newborn records linked to a mother's maternal health record
21	MALNUTRITION	Malnutrition assessment records (height, Body Mass Index, type)

Table 3.13: 21 Normalized MySQL Database Tables

3.7.1 Sample Normalized Tables (1NF to 3NF)

To illustrate how the normalization principles described in Section 3.6.2 apply across the schema, this subsection presents five representative entities from the implemented database, populated with sample Bangladeshi health-sector data and accompanied by an explanation of how each satisfies First, Second, and Third Normal Form.

HealthFacility

Keys: FacilityID (Primary Key), RegionID (Foreign Key)

FacilityID	FacilityName	FacilityType	RegionID
6	Dhaka Hospital, icddr,b	Hospital	1
7	Dobir Uddin Hospital, Kasiadanga, Rajshahi	Hospital	3
8	Doctors Care Clinic and Hospital, Barguna, Barisal	Hospital	5
10	Dr.Khadem Hossain Clinic, Bangla Bazar, Barisal	Health Facility	5
11	Dream Hospital, Begumganj, Noakhali	Hospital	2

Table 3.14: HealthFacility – Sample Normalized Data

Normalization Explanation (up to 3NF): HealthFacility satisfies First Normal Form because FacilityID, FacilityType, FacilityName, and RegionID contain atomic values and each row is uniquely identified by FacilityID. It satisfies Second Normal Form because the single-column primary key FacilityID fully determines all three non-key attributes, so no partial dependency is possible. Third Normal Form is achieved by storing no RegionName, Population, or RegionType in this table; RegionID references AdministrativeRegion, keeping regional facts in one place and preventing transitive update anomalies.

Table Content Summary: This populated registry contains 180 healthcare facilities. The final database includes categories such as Government District/General Hospital, Government Medical College Hospital, Health Facility, Hospital, and Public Health Institute. The five displayed records, FacilityID 6, 7, 8, 10, and 11, are actual rows from the final SQL dump and are linked to valid AdministrativeRegion records through RegionID.

HospitalBed

Keys: BedID (Primary Key), FacilityID (Foreign Key)

BedID	BedType	Status	FacilityID
6	Emergency	Available	6
7	General	Available	7
8	ICU	Available	8
10	General	Available	10
11	ICU	Available	11

Table 3.15: HospitalBed – Sample Normalized Data

Normalization Explanation (up to 3NF): HospitalBed satisfies First Normal Form because BedID, FacilityID, BedType, and Status each store a single scalar value. It satisfies Second Normal Form because the single-column primary key BedID fully determines FacilityID, BedType, and Status. Third Normal Form is maintained by excluding facility name, type, region, and other facility-level facts; FacilityID alone references HealthFacility, preventing those details from being repeated in every bed record.

Table Content Summary: The table contains 60 populated inpatient-bed records. Its implemented bed categories are Emergency, General, and ICU, while Status uses Available or Occupied. The sample shows BedID 6, 7, 8, 10, and 11, each connected to a valid facility through FacilityID.

Designation

Keys: DesignationID (Primary Key)

DesignationID	DesignationName
7	Professor
14	Assistant surgeon/OSD/equivalent
15	Sub Assistant Community Medical Officer (SACMO)
17	Health Assistant
19	Medical Officer

Table 3.16: Designation – Sample Normalized Data

Normalization Explanation (up to 3NF): Designation contains atomic DesignationID and DesignationName values. The single-column primary key DesignationID fully determines DesignationName, so partial dependency cannot occur. Because the table has only one non-key attribute and no non-key-to-non-key dependency, it also satisfies Third Normal Form without decomposition.

Table Content Summary: This 31-row reference table standardizes healthcare job titles and cadre posts used by the populated database. It includes academic, clinical, administrative, field, and technical roles such as Professor, Assistant Surgeon, SACMO, Health Assistant, Medical Officer, Health Inspector, and multiple Medical Technologist disciplines. The displayed DesignationID values are actual database records.

Disease

Keys: DiseaseID (Primary Key)

DiseaseID	DiseaseName	ICDCode
14	Dengue	A90
15	COVID-19	U07.1
16	Measles	B05.9
19	Cholera (Diarrheal Infection)	A00.9
20	Tuberculosis (Pulmonary TB)	A15.0

Table 3.17: Disease – Sample Normalized Data

Normalization Explanation (up to 3NF): Disease stores one atomic DiseaseID, DiseaseName, and ICDCode per row. DiseaseID is the single-column primary key and fully determines both non-key attributes, so the relation has no partial dependency and satisfies Second Normal Form. No non-key attribute determines another non-key attribute; disease catalog data is also separated from patient, case, facility, and treatment facts. The physical Disease table therefore satisfies Third Normal Form.

Table Content Summary: The database contains nine fully populated disease master records, each with an ICDCode. The displayed rows are DiseaseID 14, 15, 16, 19, and 20 for Dengue, COVID-19, Measles, Cholera, and Pulmonary Tuberculosis. These values come directly from the final SQL dump and provide standardized disease references for the related case tables.

HealthWorker

Keys: WorkID (Primary Key), FacilityID (Foreign Key), DesignationID (Foreign Key)

WorkID	WorkerName	FacilityID	DesignationID	Gender
6	Dr. Ahsan Rahman	6	19	Male
7	Dr. Nusrat Sultana	7	19	Female
8	Dr. Farzana Ahmed	8	19	Female
10	Dr. Tanvir Hasan	10	19	Male
11	Dr. Sadia Rahman	11	19	Female

Table 3.18: HealthWorker – Sample Normalized Data

Normalization Explanation (up to 3NF): HealthWorker satisfies First Normal Form because WorkID, WorkerName, FacilityID, DesignationID, and Gender are atomic. It satisfies Second Normal Form because WorkID is a single-column primary key that fully determines every other attribute. Third Normal Form is preserved by keeping facility and designation descriptions in HealthFacility and Designation and retaining only their foreign keys. RegionID is not duplicated in HealthWorker; a worker's region is derived through HealthWorker -> HealthFacility -> AdministrativeRegion.

Table Content Summary: This table contains 12 populated medical and health-worker deployment records. Each worker is linked to one valid facility and one official designation, while Gender remains a direct worker attribute. The five displayed rows, WorkID 6, 7, 8, 10, and 11, are actual records from the final database; their regional assignment is obtained through the linked facility.

Relational Schema and Normalization Compliance Summary:

- **First Normal Form:** All tables feature unique record identifiers and atomic scalar column values with no repeating groups.
- **Second Normal Form:** All non-key attributes are fully dependent on their primary key, eliminating partial functional dependencies.
- **Third Normal Form:** Transitive dependencies are eliminated across all entities by abstracting reference data into dedicated lookup tables linked via foreign keys.

3.8 Logical Entity Relationship Diagram Development

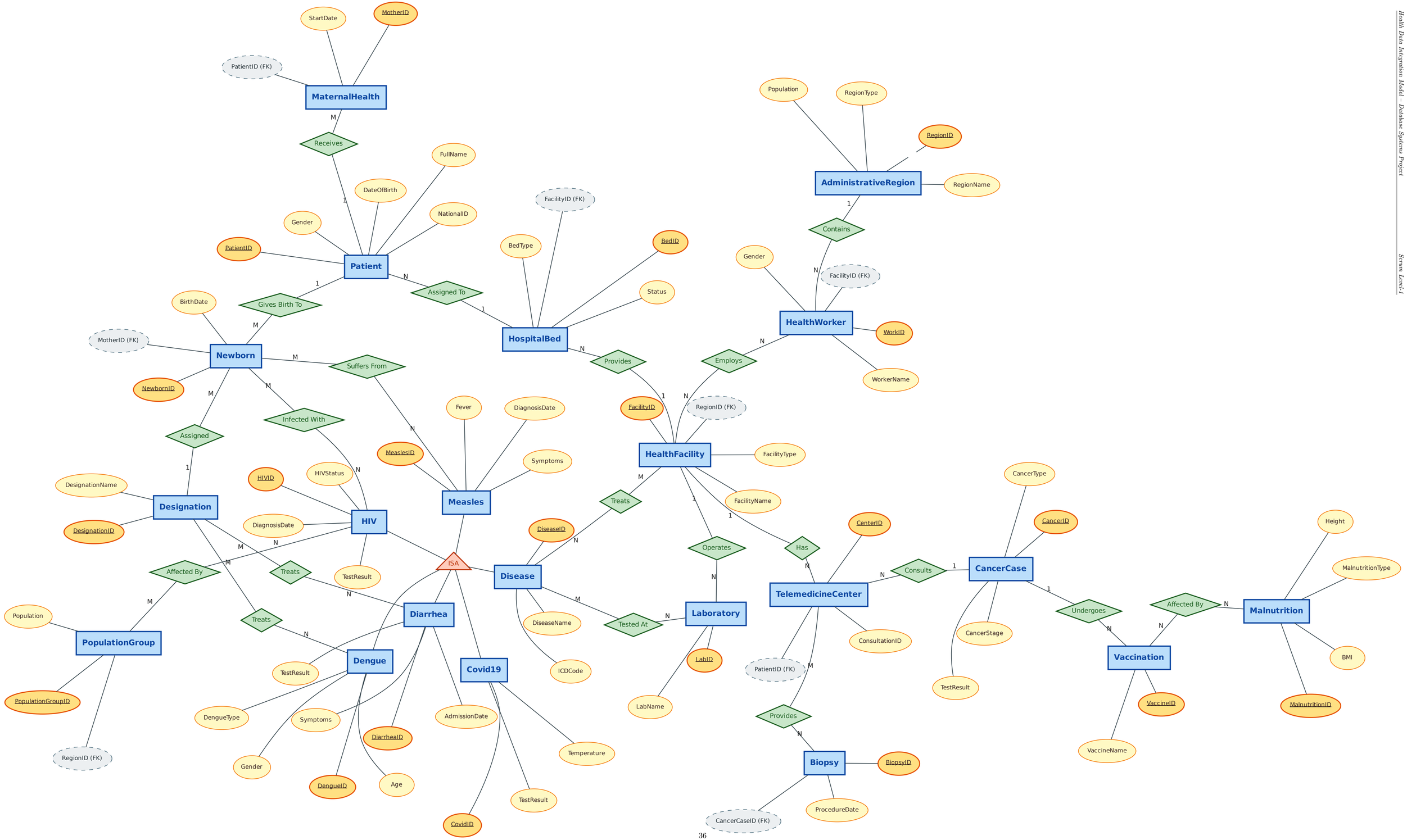
The logical ERD identifies exactly 21 healthcare entities from the reviewed source material, and all 21 are implemented as physical MySQL tables. The diagram communicates business relationships and cardinalities; the SQL schema realizes those relationships with 25 foreign-key constraints. Exact physical attributes and foreign-key paths are documented in Section 3.9.

21 Implemented Entities			
Patient	HealthFacility	Disease	CancerCase
HealthWorker	Laboratory	TelemedicineCenter	HospitalBed
Vaccination	MaternalHealth	Newborn	Malnutrition
AdministrativeRegion	PopulationGroup	Designation	Biopsy
Dengue	Covid19	Measles	HIV
Diarrhea			

Table 3.19: 21 Entities Shared by Logical ERD and Physical Database

Description: The logical diagram and the MySQL implementation use the same 21 entity names. Relationship diamonds and cardinalities express the conceptual view; the physical schema implements those links through the foreign-key columns documented in Section 3.9.

Entity Relationship Diagram - Project: Health (Group: NULL TERMINATORS)



3.9 Entity-Relationship (ER) Diagram Description

The ERD models administrative regions, healthcare facilities, resources, personnel, patients, and disease or condition records. The graphical diagram is the conceptual view; the attribute lists below reproduce the authoritative physical MySQL columns so the report, ERD interpretation, and database queries remain consistent.

3.9.1 Core, Service, and Population Entities

AdministrativeRegion: RegionID (PK), RegionName, Population, RegionType.

HealthFacility: FacilityID (PK), FacilityType, FacilityName, RegionID (FK).

HealthWorker: WorkID (PK), WorkerName, FacilityID (FK), DesignationID (FK), Gender.

Laboratory: LabID (PK), LabName, FacilityID (FK).

HospitalBed: BedID (PK), FacilityID (FK), BedType, Status.

Patient: PatientID (PK), FullName, DateOfBirth, Gender, NationalID, BedID (FK).

MaternalHealth: MotherID (PK), PatientID (FK), StartDate.

Newborn: NewbornID (PK), MotherID (FK), BirthDate.

TelemedicineCenter: CenterID (PK), ConsultationID, PatientID (FK).

PopulationGroup: PopulationGroupID (PK), RegionID (FK), Population.

Designation: DesignationID (PK), DesignationName.

Implementation note: All six Patient columns, both HealthWorker foreign keys, Laboratory.FacilityID, PopulationGroup.Population, and the other listed implementation attributes are present and populated in the final SQL schema.

3.9.2 Medical Conditions and Disease Tracking Entities

Malnutrition: MalnutritionID (PK), PatientID (FK), Height, BMI, MalnutritionType.

Vaccination: VaccineID (PK), VaccineName, PatientID (FK).

CancerCase: CancerID (PK), PatientID (FK), LabID (FK), CancerType, TestResult, CancerStage.

Biopsy: BiopsyID (PK), CancerCaseID (FK), ProcedureDate.

Disease: DiseaseID (PK), DiseaseName, ICDCode.

Covid19: CovidID (PK), DiseaseID (FK), PatientID (FK), TestResult, Temperature.

Measles: MeaslesID (PK), DiseaseID (FK), PatientID (FK), Symptoms, Fever, DiagnosisDate.

HIV: HIVID (PK), DiseaseID (FK), PatientID (FK), DiagnosisDate, HIVStatus, TestResult.

Dengue: DengueID (PK), DiseaseID (FK), PatientID (FK), Age, Gender, DengueType.

Diarrhea: DiarrheaID (PK), DiseaseID (FK), PatientID (FK), Symptoms, TestResult, AdmissionDate.

3.9.3 Physical Relationship Implementation

- Region links: HealthFacility.RegionID and PopulationGroup.RegionID reference AdministrativeRegion.RegionID.
- Facility links: HealthWorker.FacilityID, Laboratory.FacilityID and HospitalBed.FacilityID reference HealthFacility.FacilityID.
- Patient links: Patient.BedID references HospitalBed.BedID; MaternalHealth, TelemedicineCenter, Vaccination and Malnutrition use PatientID.
- Maternal/newborn link: Newborn.MotherID references MaternalHealth.MotherID.
- Cancer links: CancerCase references Patient and Laboratory; Biopsy.CancerCaseID references CancerCase.CancerID.
- Disease-case links: Dengue, Covid19, Measles, HIV and Diarrhea each reference Disease.DiseaseID and Patient.PatientID.
- Designation link: HealthWorker.DesignationID references Designation.DesignationID. Together these paths account for all 25 physical foreign keys.

3.10 Data Validation

After database construction, extensive validation is performed using SQL queries to verify record counts, relationship consistency, aggregate totals, and statistical accuracy. Results are cross-checked against original source documents.

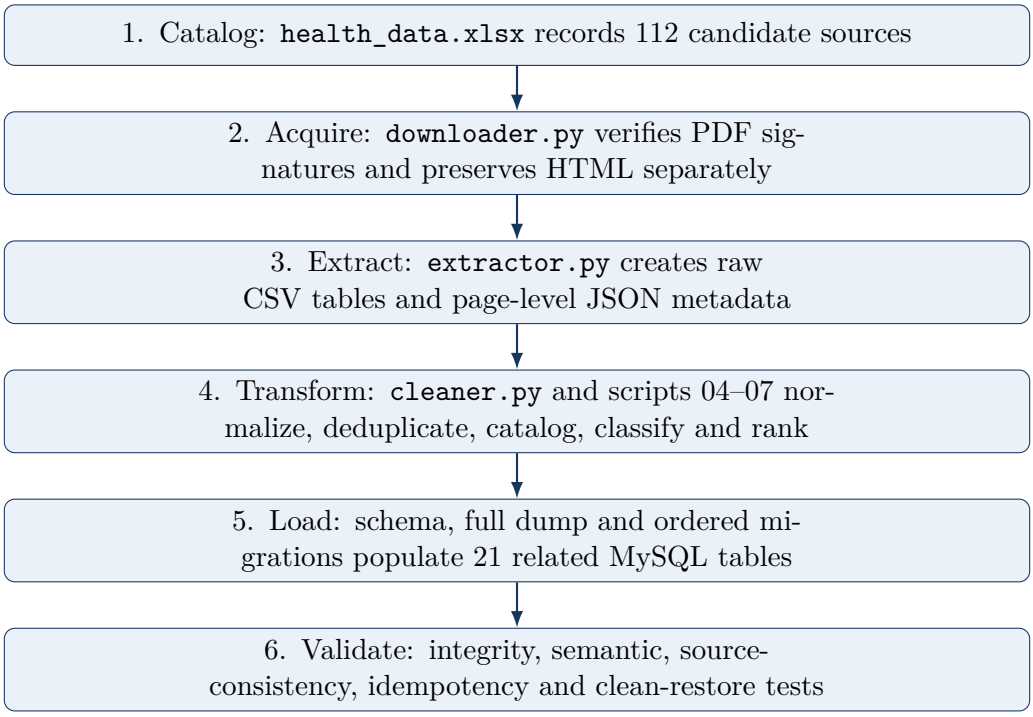
3.11 Final Sprint 1 Deliverables

1. Health Data Repository (Portable Document Format collection)
2. Automated Portable Document Format Collection System (Python scripts)
3. Extracted Comma-Separated Values Dataset Collection

4. Data Cleaning and Transformation Scripts
5. MySQL Database with 21 normalized tables
6. Relationship and Entity Documentation
7. Conceptual (Logical) Entity Relationship Diagram (21 Entities)
8. Physical Database Entity Relationship Diagram (21 Tables)
9. Project Workflow Documentation

3.12 End-to-End ETL Pipeline

The repository implements a traceable Extract-Transform-Load pipeline in which every stage has an input, a defined operation, an output and a verification artifact.



Stage	Program/artifact	Main operation	Auditable output
Extract	`downloader.py`	Reads the clean catalog; retries retrieval; verifies the `%PDF-` signature; stores HTML truthfully.	`pdfs/`, `source_pages/`; failure report when generated.
Extract	`extractor.py`	Reads up to 150 pages per PDF; keeps tables with at least 5 rows and 2 columns; removes repeated table hashes within a document.	Raw CSVs under `extracted_tables/`; page, size and content hash in `metadata/`.
Transform	`cleaner.py`	Normalizes whitespace, null tokens and column names; removes empty rows or columns and duplicate rows without replacing raw evidence.	Regenerable cleaned tables and committed cleaning-summary report.
Transform	scripts 04–07	Detects duplicate files, catalogs dimensions, applies transparent keyword classes and ranks candidate tables.	Duplicate, catalog, classification and best-table reports under `reports/`.

Stage	Program/artifact	Main operation	Auditable output
Load	Schema, migrations and full dump	Creates 21 tables, enforces 89 mandatory columns and 25 foreign keys, and loads the canonical 495-row snapshot.	`sql/schema.sql`, `sql/migrations/`, full SQL dump.
Validate	`validation_queries.sql`	Tests structure, row counts, every foreign-key path, blanks, duplicates, semantic rules and source counts; repeats after a clean restore.	Final validation, restore validation and migration-result reports under `reports/`.

3.12.1 Reproduction Commands

```
python -m pip install -r requirements.txt
python scripts/downloader.py
python scripts/extractor.py
python scripts/cleaner.py
python scripts/04_find_duplicates.py
python scripts/05_build_catalog.py
python scripts/06_classify_tables.py
python scripts/07_find_best_tables.py

mysql -u healthuser -p < sql/health_db_21_tables_with_data.sql
mysql -u healthuser -p < sql/validation/validation_queries.sql
```

The public repository is https://github.com/Ratul-debug/Health_DB_Project. The link opens the repository directly. No database password is stored in the repository.

3.13 ETL and Database Tests, Results and Findings

The following results are supported by the committed validation outputs.

ID	Test	Expected	Actual result/evidence	Status
T01	Schema structure	21 tables, 21 PKs, 25 FKs	Counts are 21, 21 and 25	Pass
T02	Mandatory columns	No nullable implementation column	89 columns; nullable count 0	Pass
T03	Population completeness	No empty table	495 total rows; empty table count 0	Pass
T04	Referential integrity	Zero orphan rows in all FK paths	All 25 relationship checks return 0	Pass
T05	NULL/blank scan	Zero stored NULL or blank value	NULL/blank issue count is 0	Pass
T06	Semantic quality	Every documented issue count equals 0	Region, gender, ICD, stage and BMI checks return 0	Pass
T07	Source-backed expansion	80 facilities, 8 IPH labs and 12 designations match Health Bulletin 2023	All three mismatch checks return 0	Pass
T08	Normalized duplicates	No duplicate facility, laboratory or designation name	All three duplicate checks return 0	Pass
T09	Migration idempotency	Re-running migration 005 adds nothing	`added_count = 0` after re-run	Pass
T10	Restore reproducibility	Fresh restore equals live validation	Both reports have 156 lines and are byte-for-byte identical	Pass

3.13.1 Findings

1. The canonical dump restores to the same validated state as the live database.
2. Foreign-key enforcement and validation agree; no relationship path contains an orphan row.
3. Source-backed loading is limited to evidence available at the required granularity; aggregate bed totals are not converted into individual beds.
4. The database is a demonstration dataset rather than a clinical registry; synthetic patient and event rows are used only for schema and query testing.
5. The team completed internal checks, and Miskatul Anwar, the assigned senior mentor, technically reviewed and approved the ETL work. The supervisor is the final approver.

3.14 ETL Technical Review and Approval Status

The team prepared and internally checked the pipeline. Miskatul Anwar, the senior mentor assigned by the supervisor, observed the work and provided ETL technical approval. Professor Dr. Rudra Pratap Deb Nath is the final supervisor approver. The table below records each participant's role and review status; formal signatures are collected once on the Group Details page.

Person	Role	Review status
Md. Alamin Molla	Team member	Prepared / internally checked
Sheikh Showkotul Islam Shakib	Team member	Prepared / internally checked
Ratul Hasan Nipu	Team member	Prepared / internally checked
Jahid Alam Ridoy	Team member	Prepared / internally checked
Miskatul Anwar	Assigned senior mentor / ETL technical approver	Technically reviewed and approved
Professor Dr. Rudra Pratap Deb Nath	Final supervisor approver	Final approval

3.15 Data Provenance and Traceability

Provenance is maintained as an evidence chain rather than a single URL. Each database value can be traced through the following repository layers.

Layer	Evidence artifact	Provenance captured	Purpose
Discovery	`health_data.xlsx`	Catalog row, publisher, subcategory, dataset name, URL, source type and quality note	Establishes why and where a source was discovered.
Archive	`pdfs/`, `source_pages/`	Retained source bytes and truthful response type	Prevents HTML from being presented as PDF evidence.
Extraction	`metadata/*.json`, `extracted_tables/`	Original document, page, CSV filename, dimensions and content hash	Links a raw table to its document location.
Transform	Cleaning, catalog and classification reports	Normalization, dimensions, duplicate relationship, keyword class and rank	Records screening without replacing raw evidence.
Load	SQL migrations and full dump	Inserted values, destination table, keys and migration order	Reproduces the physical database state.
Validation	Final, restore and migration-result reports	Structure, row counts, FK, blank, semantic and source-count checks	Shows that the loaded state satisfies declared rules.

Chapter 4

Deployment and Maintenance

4.1 Repository Structure

All Extract-Transform-Load pipeline code, data-cleaning scripts, extracted Comma-Separated Values datasets, SQL files, validation reports, and documentation are deployed to a public GitHub repository.

- **Repository Web Address:** https://github.com/Ratul-debug/Health_DB_Project

The final repository is organized into the following directories and project files:

- `/extracted_tables/ and /metadata/:` raw tables and extraction records.
- `/pdfs/ and /source_pages/:` valid PDF and HTML source files.
- `/scripts/:` download, extraction, cleaning, and catalog scripts.
- `/sql/:` canonical schema, full data dump, migrations, and validation queries.
- `/reports/:` validation, restore, cleanup, and expansion results.
- **Project files:** health_data.xlsx, ERD files, report, README, and alignment documents.

4.2 How to Run the Pipeline

1. Clone the repository from GitHub and enter the Health_DB_Project directory.
2. Install the packages: `python -m pip install -r requirements.txt`
3. Optional source processing: run downloader.py, extractor.py, cleaner.py, and scripts 04-07.
4. Restore the final database: `mysql -u healthuser -p < sql/health_db_21_tables_with_data.sql`
5. Validate it: `mysql -u healthuser -p < sql/validation/validation_queries.sql`
6. Use migrations 001-005 only for an earlier database; the full dump already includes them.

4.3 Maintenance

The database and scripts are designed for incremental updates while preserving the submitted 21-entity structure. New compatible government reports can be added to the source catalog and reprocessed without changing the established entity or attribute names.

- Back up health_db before any migration or data update and retain the numbered migration files as the reproducible history.
- Update health_data.xlsx and the source archive together. PDF responses remain in pdfs/, while HTML responses remain in source_pages/ with their real file types.
- Preserve extracted_tables/ and metadata/ as raw evidence and record every curated or synthetic value in DATA_PROVENANCE.md.
- After an approved change, regenerate sql/schema.sql and sql/health_db_21_tables_with_data.sql, then rerun sql/validation/validation_queries.sql.
- Restore the full dump into a separate test database and compare reports/restore_test_validation.txt with reports/final_validation.txt before committing.
- Commit and push only after validation, then verify that local and remote master and 21-entity-final branches resolve to the same commit.

The final canonical snapshot contains 21 populated tables, 89 mandatory columns, 21 primary keys, 25 foreign-key constraints, and 495 demonstration rows. No table is empty, and every NULL/blank, orphan, duplicate, and documented semantic issue count is zero.

The canonical dump and the clean restore produce identical 156-line validation reports. At final verification, master and 21-entity-final are synchronized to the same commit.

Chapter 5

Collaboration

5.1 Intra-Group Collaboration

Within our group, collaboration is managed through regular meetings, including daily and monthly progress check-ins, and a shared GitHub codebase. Task assignments are tracked using the sprint backlog, and progress is reviewed at the end of each work session to reduce collaboration redundancy and keep meeting time focused.

A key focus of this intra-group collaboration is coordinating with the **Obsidians**, who are working on the same database project. Since both of our teams are independently developing different parts of the database, we communicate closely to identify overlapping entities and attributes. Although this cross-team collaboration is targeted, these periodic discussions are crucial to minimize data redundancy, avoid designing duplicate attributes or similar components, and ensure seamless consistency and compatibility between our two databases.

Overall, intra-group collaboration is productive. Splitting responsibilities by role, team leadership, data collection, Entity Relationship Diagram design, and script writing, means that each member can work in parallel on a well-defined piece of the project while still depending on the others' outputs, which keeps everyone engaged with the bigger picture. The most challenging moments arise during the data cleaning phase, when extracted Comma-Separated Values (CSV) files from different sources do not follow the same column structure; reconciling these requires several back-and-forth discussions and a shared agreement on naming conventions before the database design can proceed. Regular short check-ins help catch these mismatches and cross-team entity overlaps early, rather than after the schema has already been finalized.

5.2 Inter-Group Collaboration

At the time of writing this report, the inter-level Scrum of Scrum sessions with the other sector groups have not yet taken place. Our group has so far focused on completing the Healthcare sector deliverables independently, and inter-group integration with the other six MODELBD sectors (Agriculture, Energy, and others) is expected to begin in a subsequent phase of the course. This section will be expanded with our experiences once those sessions occur.

5.3 Good Memory

The most rewarding moment of Sprint 1 is watching the first successful end-to-end run of the pipeline: a batch of previously unreadable Portable Document Format health bulletins goes in, and clean, query-able rows land in MySQL tables that match our normalized schema. It confirms that the whole chain, from messy government PDFs to a working relational database, actually holds together.

5.4 Bad Memory

The most difficult stretch is during the data cleaning phase (Week 4), when we discover that Comma-Separated Values files extracted from different cancer-registry reports use inconsistent column orders and naming for the same fields. It takes several rounds of manual comparison and a team discussion to agree on a single standardized schema before we can safely merge the datasets.

Chapter 6

Comments on Course Pedagogy

Aspect	Opportunity / Strength	Limitation / Suggestion
Scrum Methodology	Provides real industry experience in agile project management	Initial learning curve for students unfamiliar with Scrum
Collaborative Development	Working on a shared system builds teamwork and integration skills	Coordination overhead increases with group size
Real Data Usage	Using actual government data makes the project meaningful	Real data is messy and requires significant preprocessing time
Industry Relevance	Extract-Transform-Load pipelines, version control, and normalization are directly applicable to industry	A short introductory session on Git/GitHub workflows early in the course would help students who have not used version control collaboratively before
Data Quality & Performance	Working with real, messy government data gives hands-on experience in validation, integrity-checking, and query optimization, skills directly transferable to industry data engineering roles	Proper data validation and database optimization techniques should be applied more systematically (for example, automated constraint checks, indexing strategy) to improve long-term system reliability and performance

Table 6.1: Course Pedagogy: Opportunities and Limitations

6.1 Lessons Learned

Through this project, we gain practical experience in several areas beyond standard coursework:

- **Skills Learned:** Git and GitHub (version control and collaborative workflows), Python (scripting for automation and data processing), Extract-Transform-Load

pipeline development, and Scrum (agile project management), all applied at the scale of a complete, working "tiny project" delivered end-to-end.

- **Industry Processes:** Real-world data is rarely structured. Extract-Transform-Load pipelines are fundamental to any data-driven system.
- **Collaboration:** Clear task division and version control are essential when multiple people work on the same codebase.
- **Conflict Resolution:** Differences in naming conventions and data interpretations require negotiation and consensus.
- **Management:** Sprint-based planning helps keep the project on track despite the large volume of work.
- **Database Design:** We learn practical database design using Entity Relationship diagrams, translating real-world health entities and relationships into a normalized schema.
- **SQL Proficiency:** Our SQL querying and database management skills improve substantially through writing validation queries, joins, and integrity checks.
- **Teamwork and Task Distribution:** Dividing tasks across data collection, extraction, cleaning, and database design gives us direct experience in team-based software delivery.
- **Problem-Solving:** Implementing the database, from schema design to handling malformed data, strengthens our practical problem-solving skills.
- **Documentation and Presentation:** Preparing this report and the accompanying ERDs enhances our ability to document and present technical work clearly.

6.2 Suggestions for Future Batches

- Providing students with a sample cleaned dataset in the first week would reduce time spent on data quality issues and allow more focus on database design.
- At the beginning of the project, a clear and detailed guideline should be provided by the course teacher or project mentor regarding the project objectives, requirements, and expected outcomes.
- Sufficient time should be allocated for project planning, database design, implementation, testing, and documentation.
- Regular consultation sessions with the mentor can help students overcome challenges more effectively.

- Sharing a list of known-reliable government data sources at the start of the course would reduce the time groups spend on broken links and unreliable mirrors during data discovery.
- Scheduling the inter-level Scrum of Scrum sessions earlier, in parallel with Sprint 1 rather than strictly after it, would let groups align on shared entities (such as `District`/`Division`) before each group finalizes its own schema independently.
- Future versions of the project may include a graphical user interface, enhanced security features, and advanced reporting capabilities.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This project addresses the problem of inaccessible and unstructured health data from Bangladesh government sources. The developed solution is a fully normalized MySQL database comprising 21 relational tables covering the core entities of the Bangladesh healthcare ecosystem, supported by automated Extract-Transform-Load pipeline scripts and comprehensive ERDs.

Starting from over 100 scattered Portable Document Format documents and ending with a structured, validated, and queryable relational database, the project demonstrates that systematic data engineering, including automated collection, extraction, cleaning, normalization, and validation, can transform unstructured government data into a professionally usable information system.

The significance of this work lies in its potential to support health administration, policy analysis, research, and reporting at national and district levels within Bangladesh.

7.2 Limitations

- The current database covers data from the reports collected during Sprint 1 only; not all government health documents are included.
- Scanned Portable Document Format pages could not be processed using standard text extraction methods.
- The system does not yet include a user interface or Application Programming Interface layer for non-technical users.

7.3 Future Work

- Extend coverage to additional government health reports and time periods.
- Implement Optical Character Recognition-based extraction for scanned Portable Document Format documents.
- Automate periodic data updates by monitoring government portals.

- Develop analytical dashboards for visualization of health trends.
- Integrate with the MODELBD system at the inter-sector level.
- Expose the database through a Representational State Transfer Application Programming Interface for programmatic access.

Chapter 8

Bibliography

1. Ministry of Health and Family Welfare, Bangladesh. *Health Bulletin.* Directorate General of Health Services, various years.
2. Directorate General of Health Services, Bangladesh. *Annual Report.* Ministry of Health and Family Welfare, various years.
3. Bangladesh Bureau of Statistics. *Statistical Yearbook of Bangladesh.* Ministry of Planning, Government of the People's Republic of Bangladesh.
4. Directorate General of Health Services, and National Institute of Cancer Research and Hospital, Bangladesh. *National Cancer Control Strategy and Plan of Action 2009–2015.* Ministry of Health and Family Welfare, Government of the People's Republic of Bangladesh, 2008.
5. R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems.* Pearson, seventh edition.
6. A. Silberschatz, H. F. Korth, and S. Sudarshan. *Database System Concepts.* McGraw-Hill, seventh edition.
7. W. McKinney. *Python for Data Analysis.* O'Reilly Media, third edition.
8. United Nations International Children's Emergency Fund, Bangladesh. *UNICEF Bangladesh Annual Report 2025: Delivering Results for Children Through a Difficult Year.* UNICEF, 2025.
9. Management Information System (MIS) Unit, Directorate General of Health Services, Bangladesh. *District Health Information Software 2 (DHIS2) Platform: National Routine Health Data Warehouse.* Ministry of Health and Family Welfare, Government of the People's Republic of Bangladesh.
10. Directorate General of Health Services, Bangladesh. *National Health Facility Registry Data Portal.* Management Information System (MIS) Unit, Ministry of Health and Family Welfare.
11. National Cervical and Breast Cancer Screening Programme (NCBCSP), Bangladesh. *Case-Based Cervical Cancer Screening Registry (DHIS2 Tracker Electronic Repository).* Directorate General of Health Services, Ministry of Health and Family Welfare.